

SOFTWARE REQUIREMENTS SPECIFICATION & WORK BREAKDOWN STRUCTURE

Trip Expense Simplification Engine

Problem Statement #37 — Group Travel Expense Simplification Engine

Document	Software Requirements Specification (Part A) and Work Breakdown Structure (Part B)
Version / status	1.0 — Draft for review
Date	October 02, 2026
Actors	Group Traveler (primary), Trip Admin (secondary), Currency Service (external)
Basis	Use-case flow “Compute Debt Settlement”, Use Case Diagram, Requirements Table (FR-001-005, NFR-001-002 given)
Companion files	Requirements_RTM_Architecture.docx, Architecture_Diagram.png

Note: Requirements FR-006-013 and NFR-003-011, all numeric targets for them, and every WBS effort figure are proposals derived from the supplied material and should be confirmed by the project team.

Part A — Software Requirements Specification

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the Trip Expense Simplification Engine. It is intended for the project team (developers, testers, designers), the product owner and reviewers, and serves as the baseline for design, implementation, testing and acceptance.

1.2 Scope

The Trip Expense Simplification Engine lets a group of travelers record shared expenses (including uneven splits and foreign-currency expenses), see each person's net balance, and receive the smallest practical set of peer-to-peer payments that brings every balance to zero. A Trip Admin manages who belongs to the trip. The system notifies travelers about settlement events and tolerates an unavailable exchange-rate service by falling back to cached rates.

- **In scope:** expense entry with custom splits, currency normalization to a trip base currency, net-balance computation, min-flow settlement plan, payment confirmation tracking, trip membership management, notifications, audit trail.
- **Out of scope (assumed):** moving real money (payments happen outside the system and are only recorded), expense editing/deletion, receipt scanning, tax handling, multi-trip global balances.

1.3 Definitions, acronyms and abbreviations

Term	Definition
Base currency	The single currency in which all trip expenses and balances are normalized.
Net balance	For a traveler: total amount paid minus total owed share of expenses, adjusted by confirmed settlement payments. Positive = is owed money; negative = owes money.
Settlement plan / graph	The set of directed payment edges (payer → payee, amount) that brings all net balances to zero.
Settlement edge	One payment in the settlement plan; has status Proposed or Paid.
Min-flow	Debt simplification that minimizes the number of payment transactions needed to settle all balances.
Estimated expense	An expense converted with a cached (not live) exchange rate because the rate service was unreachable.
FR / NFR / RTM	Functional Requirement / Non-Functional Requirement / Requirements Traceability Matrix.
WBS	Work Breakdown Structure.

1.4 References

- Use-Case Flow: Compute Debt Settlement (UseCase_Flow_ComputeSettlement.docx)
- Use Case Diagram — Trip Expense System (UseCaseDiagram.pdf)
- Requirements Table, Problem Statement #37 (RequirementsTable.docx)
- Requirements, RTM and Architecture pack (Requirements_RTM_Architecture.docx)

1.5 Document overview

Section 2 describes the product in context; Section 3 describes the use cases; Section 4 lists specific requirements, interfaces, data and business rules; Section 5 summarizes the architecture; Section 6 gives the traceability matrix; Section 7 lists open issues. Appendix A explains the settlement algorithm. Part B contains the Work Breakdown Structure, schedule and risks.

2. Overall Description

2.1 Product perspective

The system is a new, self-contained web and mobile-responsive application backed by a set of services. It depends on two external systems: an exchange-rate provider (the “Currency Service” actor) and an email / push messaging provider. It does not integrate with banks or payment networks.

2.2 Product functions

Use case	Summary
Add Expense	Group Traveler records a shared expense and its split among selected members; includes Convert Currency.
Convert Currency	Converts a foreign-currency expense to the base currency using the exchange rate of the day (Currency Service).
Compute Debt Settlement	Computes net balances and the minimal payment plan; travelers review it and confirm payments; extended by Notify Travelers.
Notify Travelers	Informs travelers of plan creation, payment confirmation and plan changes.
Manage Trip Travelers	Trip Admin adds or removes travelers.
View Balance	Group Traveler sees their real-time net balance.

2.3 User classes and characteristics

User / actor	Description	Key needs
Group Traveler (primary)	Any member of a trip; typically non-technical and using a phone while travelling.	Quick expense entry, trustworthy balances, a clear short list of who pays whom.
Trip Admin (secondary)	A traveler with membership-management rights.	Add / remove travelers safely without corrupting balances.
Currency Service (external)	Third-party daily exchange-rate API.	Well-formed requests; tolerant handling of outages.

2.4 Operating environment

- Client: latest two versions of Chrome, Safari, Firefox and Edge; screen width from 360 px.
- Server: containerized stateless services behind an API gateway; PostgreSQL, Redis and a message queue as supporting infrastructure.
- Network: HTTPS (TLS 1.2 or higher) for all client and external traffic.

2.5 Design and implementation constraints

- Settlement for up to 50 travelers and 500 expenses must complete in under 100 ms (NFR-001).
- Financial data must be encrypted at rest (AES-256) and in transit (TLS 1.2+) (NFR-002).
- Monetary values use fixed-point decimals in minor units; floating point is not permitted (NFR-005).

2.6 Assumptions and dependencies

- Payments between travelers occur outside the system; the system only records confirmation.
- Each trip has exactly one base currency, chosen when the trip is created.
- The exchange-rate provider publishes one rate per currency per day; the exact provider is still to be chosen.
- All travelers have an email address or device capable of receiving notifications.

3. Use-Case Descriptions

3.1 Compute Debt Settlement (from the supplied use-case flow)

Item	Description
Actors	Primary: Group Traveler. Secondary: Trip Admin.
Preconditions	Trip has at least two Group Travelers; at least one expense is logged; all foreign-currency expenses are already converted to the base currency (Convert Currency).
Success postcondition	A minimal set of peer-to-peer payments is generated such that every traveler's net balance equals zero after all are executed.
Failure postcondition	No settlement graph is generated and previous balances remain unchanged.

Main success scenario

1. Group Traveler selects “Settle Up”. The system displays all recorded expenses and current net balances. (FR-007)
2. The system aggregates all expense shares and payments to compute each traveler's net balance in the base currency. (FR-008)
3. The system runs the min-flow debt settlement algorithm to compute the smallest set of payments that brings every balance to zero. (FR-001)
4. The system displays the settlement plan: who pays whom and how much. (FR-007)
5. Group Traveler reviews the proposed plan.
6. Group Traveler confirms one or more payments as completed. (FR-006)
7. The system updates each traveler's balance and marks the corresponding edge as paid. (FR-006)
8. When all edges are paid, the system confirms the trip is fully settled; the use case ends. (FR-012)

Alternate flow 3a — Currency conversion data unavailable

- 3a1. The system detects that one or more expenses cannot be normalized because the exchange-rate service is unreachable.
- 3a2. The system falls back to the last successfully cached rate and flags the affected expenses as “estimated”. (FR-009)
- 3a3. The system proceeds with the calculation and shows a warning banner. (FR-009)
- 3a4. When live rates return, the system recalculates the settlement and notifies affected travelers of any change; the use case resumes at step 4. (FR-010, FR-011)

3.2 Other use cases (derived — flows not supplied)

The diagram shows these use cases but the supplied flow does not detail them. The brief descriptions below are proposals and should be confirmed.

Use case	Proposed main flow	Requirements
Add Expense	Traveler enters description, amount, currency, payer and participants; chooses equal, custom-amount or percentage split; system validates that shares sum to the total, converts currency if needed, stores the expense and updates balances.	FR-002, FR-003, NFR-005
Convert Currency	System looks up the rate for the expense date, stores base amount and rate used alongside the original amount; on lookup failure uses the cached rate and flags the expense as estimated.	FR-003, FR-009, NFR-007
Manage Trip Travelers	Trip Admin adds a traveler by email or removes a traveler; removal is blocked unless that traveler's net balance is zero.	FR-004, NFR-006
View Balance	Traveler opens the balance screen; system shows net amount owed or owed to them, refreshed within 2 seconds of the latest change.	FR-005, FR-008, NFR-003
Notify Travelers	On plan generation, payment confirmation or recalculation change, the system sends a notification to each affected traveler.	FR-010, FR-011

4. Specific Requirements

4.1 Functional requirements

Priority colours: High / Medium / Low. "Source" shows whether the requirement was given or derived from the use-case flow.

Req ID	Description	Priority	Acceptance criteria	Rationale / source
FR-001	The system shall compute the optimal debt settlement graph using min-flow algorithms to minimize total peer-to-peer payment transactions.	High	Pass: Group balance settles to net zero with minimal payment edges; Fail: Sum of debts does not equal sum of credits.	Core value proposition — reduces the number of payments travelers must make to settle up. Source: Given
FR-002	The system shall allow a Group Traveler to add a shared expense and specify an uneven split (custom amount or percentage) across selected trip members.	High	When an expense is submitted with a custom split, each traveler's owed amount is recalculated and displayed, and the sum of all shares equals the total expense amount.	Uneven splitting is the core input needed before any settlement can be computed. Source: Given
FR-003	The system shall convert an expense entered in a foreign currency to the trip's base currency using the exchange rate published for that day.	High	When an expense is logged in a non-base currency, the converted base-currency amount is stored and displayed alongside the original amount and the rate used.	Multi-currency trips cannot be settled correctly without normalizing all expenses to one currency. Source: Given
FR-004	The system shall allow a Trip Admin to add or remove Group Travelers from a trip.	Medium	When a Trip Admin removes a traveler with a zero balance, the traveler no longer appears in the trip's member list or future expense splits.	Trip membership must be managed before expenses can be attributed correctly. Source: Given
FR-005	The system shall allow a Group Traveler to view their real-time net balance (amount owed or owed to them) for a trip.	Medium	When a traveler opens the balance screen, the displayed net amount matches the sum of their expense shares minus payments made, recalculated within 2 seconds of the last update.	Travelers need visibility into their balance before and after settlement. Source: Given
FR-006	The system shall allow a Group Traveler to confirm one or more settlement payments as completed, update each affected traveler's balance, and mark the corresponding settlement edge as paid.	High	Pass: After confirmation, payer and payee balances change by exactly the payment amount and the edge shows "Paid"; Fail: balances or edge status unchanged or inconsistent.	Closes the settlement loop and keeps balances accurate as payments happen outside the app. Source: Derived: UC steps 6-7
FR-007	The system shall display, on "Settle Up", all recorded trip expenses and each traveler's current net balance, and shall present the proposed settlement plan stating who pays whom and how much.	High	Pass: Every expense and balance is listed; each plan edge shows payer, payee and amount; each debtor's outgoing payments sum to their balance. Fail: any expense or edge missing.	Travelers must be able to review and trust the plan before paying. Source: Derived: UC steps 1, 4, 5
FR-008	The system shall aggregate all expense shares and payments to compute each traveler's net balance in the trip's base currency.	High	Pass: Net balance = total paid – total owed share for every traveler, and the sum of all net balances is zero (± 0.01). Fail: any mismatch beyond tolerance.	Net balances are the input to the settlement algorithm (FR-001). Source: Derived: UC step 2
FR-009	If the exchange-rate service is unreachable, the system shall fall back to the last successfully cached rate, flag the affected expenses as "estimated", and display a warning banner while proceeding with the settlement calculation.	High	Pass: With the rate service disabled, a settlement is still generated, affected expenses carry an "Estimated" flag and a banner is shown. Fail: settlement is blocked or estimated expenses are unflagged.	An external outage must not block settlement, but users must know the figures are provisional. Source: Derived: UC alt. flow 3a1-3a3
FR-010	When live exchange rates become available again, the system shall recalculate the settlement for estimated expenses and notify affected travelers of any change.	Medium	Pass: After service recovery, estimated flags are cleared, the settlement is recomputed and each traveler whose amount changed is notified. Fail: stale estimate retained or no notification sent.	Ensures final amounts reflect real rates and travelers are aware of changes. Source: Derived: UC alt. flow 3a4

Req ID	Description	Priority	Acceptance criteria	Rationale / source
FR-011	The system shall notify Group Travelers when a settlement plan is generated, when a payment is confirmed, and when a plan changes after recalculation.	Medium	Pass: Every affected traveler receives an in-app/email notification for each event within 60 seconds. Fail: any affected traveler not notified.	Keeps all travelers informed without manual follow-up (Notify Travelers use case). <i>Source: Derived: Use Case Diagram</i>
FR-012	The system shall confirm that a trip is fully settled once all settlement edges are marked paid.	Medium	Pass: When the last edge is marked paid, all balances are zero and the trip status is "Settled". Fail: status "Settled" while any edge is unpaid or any balance is non-zero.	Gives travelers a clear end state for the trip. <i>Source: Derived: UC step 8</i>
FR-013	The system shall verify that the trip has at least two Group Travelers, at least one expense and all expenses normalized to the base currency before settlement; if verification or calculation fails, no settlement graph shall be generated and existing balances shall remain unchanged.	Medium	Pass: Settle Up on a trip with one traveler or no expenses returns an explanatory error, no graph is stored and balances are unchanged. Fail: a graph is created or balances are altered.	Enforces the use-case preconditions and failure postcondition. <i>Source: Derived: UC pre/postconditions</i>

4.2 External interface requirements

Interface	Requirement
User interface	Responsive web / mobile layout from 360 px; screens: Manage Travelers, Add Expense, View Balance, Settle Up & plan review, Confirm Payment; visible "Estimated" markers and a warning banner when cached rates are used; WCAG 2.1 AA.
Hardware	No special hardware; standard phone, tablet or desktop browser.
Software — exchange rates	HTTPS REST call to the Currency Service for one rate per currency per day; 3 s timeout, bounded retries, circuit breaker; last good rate cached.
Software — messaging	Email / push provider (SMTP, FCM / APNs) via a notification adapter with retry on failure.
Internal APIs	REST / JSON over HTTPS through the API gateway, specified with OpenAPI; asynchronous events on the message queue (ExpenseAdded, RateUpdated, SettlementComputed, PaymentConfirmed, TravelerRemoved).
Communications	TLS 1.2 or higher on every hop; JWT-based authentication.

4.3 Non-functional requirements

Req ID	Description	Priority	Acceptance criteria	Rationale / source
NFR-001	The settlement calculation engine must execute in under 100 ms for groups of up to 50 travelers with 500 expenses.	High	Pass: Benchmarking tests confirm target latency under simulated peak load.	Ensures the app remains responsive even for large group trips. <i>Source: Given</i>
NFR-002	The system shall encrypt all stored and transmitted financial data (expenses, balances, payment details) using industry-standard encryption (e.g., AES-256 at rest, TLS 1.2+ in transit).	High	Pass: Security audit / automated scan confirms all financial data fields are encrypted at rest and all API traffic uses TLS 1.2 or higher; Fail: any financial field found in plaintext in storage or transit.	Financial and personal data must be protected against breaches given the sensitive nature of group payment information. <i>Source: Provided</i>
NFR-003	The balance view and the Settle Up screen shall load and refresh within 2 seconds (95th percentile) after any expense or payment change, under a load of 500 concurrent users.	Medium	Pass: Load test shows p95 response ≤ 2 s at 500 concurrent users; Fail: p95 exceeds 2 s.	Supports the real-time balance requirement (FR-005). <i>Source: Derived from FR-005</i>
NFR-004	The system shall be available at least 99.5% per month (excluding planned maintenance) with a recovery point objective ≤ 5 minutes and a recovery time objective ≤ 1 hour.	High	Pass: Monitoring shows $\geq 99.5\%$ uptime over 30 days; a restore drill meets RPO/RTO. Fail: either target missed.	Travelers settle up at the end of trips, often at fixed times, so downtime or data loss is costly. <i>Source: —</i>

Req ID	Description	Priority	Acceptance criteria	Rationale / source
NFR-005	All monetary amounts shall be stored as fixed-point decimals (minor units) with a single documented rounding rule; rounding residuals shall be allocated so that shares always sum to the expense total and all balances net to zero after settlement.	High	Pass: Property-based tests over 10,000 random scenarios show zero discrepancy greater than 0.01; Fail: any scenario leaves a non-zero residual.	Floating-point errors would break the "net balance equals zero" guarantee. <i>Source: —</i>
NFR-006	The system shall enforce role-based access control: only a Trip Admin may add or remove travelers, and a user may view or modify only trips of which they are a member.	High	Pass: Authorization tests show unauthorized requests return HTTP 403 and leak no trip data; Fail: any cross-trip or cross-role access succeeds.	Protects privacy and prevents unauthorized changes to membership and balances. <i>Source: —</i>
NFR-007	Calls to the external exchange-rate service shall use a timeout of 3 s, bounded retries with back-off and a circuit breaker, and the system shall keep all other functions available during an outage.	High	Pass: Fault-injection test with the service down shows add-expense, balance and settlement still work using cached rates; Fail: any core function errors.	Supports the fallback behaviour in FR-009 and avoids cascading failures. <i>Source: —</i>
NFR-008	The system shall scale horizontally to support at least 1,000 concurrent trips and 10,000 concurrent users without code changes.	Medium	Pass: Load test at target volume shows throughput grows with added instances and error rate < 1%.	Allows growth beyond the initial user base without redesign. <i>Source: —</i>
NFR-009	The system shall write an immutable audit record (actor, timestamp, before/after values) for every expense, membership and payment-status change and retain it for at least 12 months.	Medium	Pass: Sampled changes each produce a complete audit entry that cannot be edited or deleted by application users; Fail: missing or mutable entries.	Resolves disputes about who changed what in a shared-money context. <i>Source: —</i>
NFR-010	The user interface shall be responsive from 360 px screen width upwards, support the latest two versions of Chrome, Safari, Firefox and Edge, and meet WCAG 2.1 AA.	Medium	Pass: Cross-browser/device test passes on the listed targets and an accessibility scan reports no AA violations; ≥ 80% of first-time test users add an expense unaided in under 60 s.	Most travelers use phones while travelling and must be able to use the app without training. <i>Source: —</i>
NFR-011	The settlement engine and currency-conversion modules shall be isolated behind defined interfaces and maintain at least 80% automated unit-test coverage, enforced by the CI pipeline.	Medium	Pass: CI fails the build when coverage drops below 80% or interface contract tests fail.	Allows the algorithm and rate provider to be replaced or upgraded safely. <i>Source: —</i>

4.4 Data requirements (logical model)

Entity	Key attributes	Notes
Trip	trip_id, name, base_currency, status (Active / Settled), created_at	One base currency per trip.
Traveler	traveler_id, name, email	May belong to many trips.
TripMember	trip_id, traveler_id, role (Traveler / Admin), joined_at, removed_at	Removal requires zero balance (BR-04).
Expense	expense_id, trip_id, payer_id, description, original_amount, original_currency, base_amount, rate_used, is_estimated, expense_date	Amounts stored as integer minor units.
ExpenseShare	expense_id, traveler_id, share_amount (and percentage if entered)	Shares sum to base_amount (BR-01).
ExchangeRate	currency, rate_date, rate, fetched_at, source, is_cached	Last good rate retained for fallback.
SettlementEdge	edge_id, trip_id, payer_id, payee_id, amount, status (Proposed / Paid), plan_version, paid_at	Replaced when plan is recalculated.
Notification	notification_id, recipient_id, event_type, channel, sent_at, status	Event types per FR-011.
AuditRecord	audit_id, actor_id, entity, action, before, after, timestamp	Append-only; retained ≥ 12 months (NFR-009).

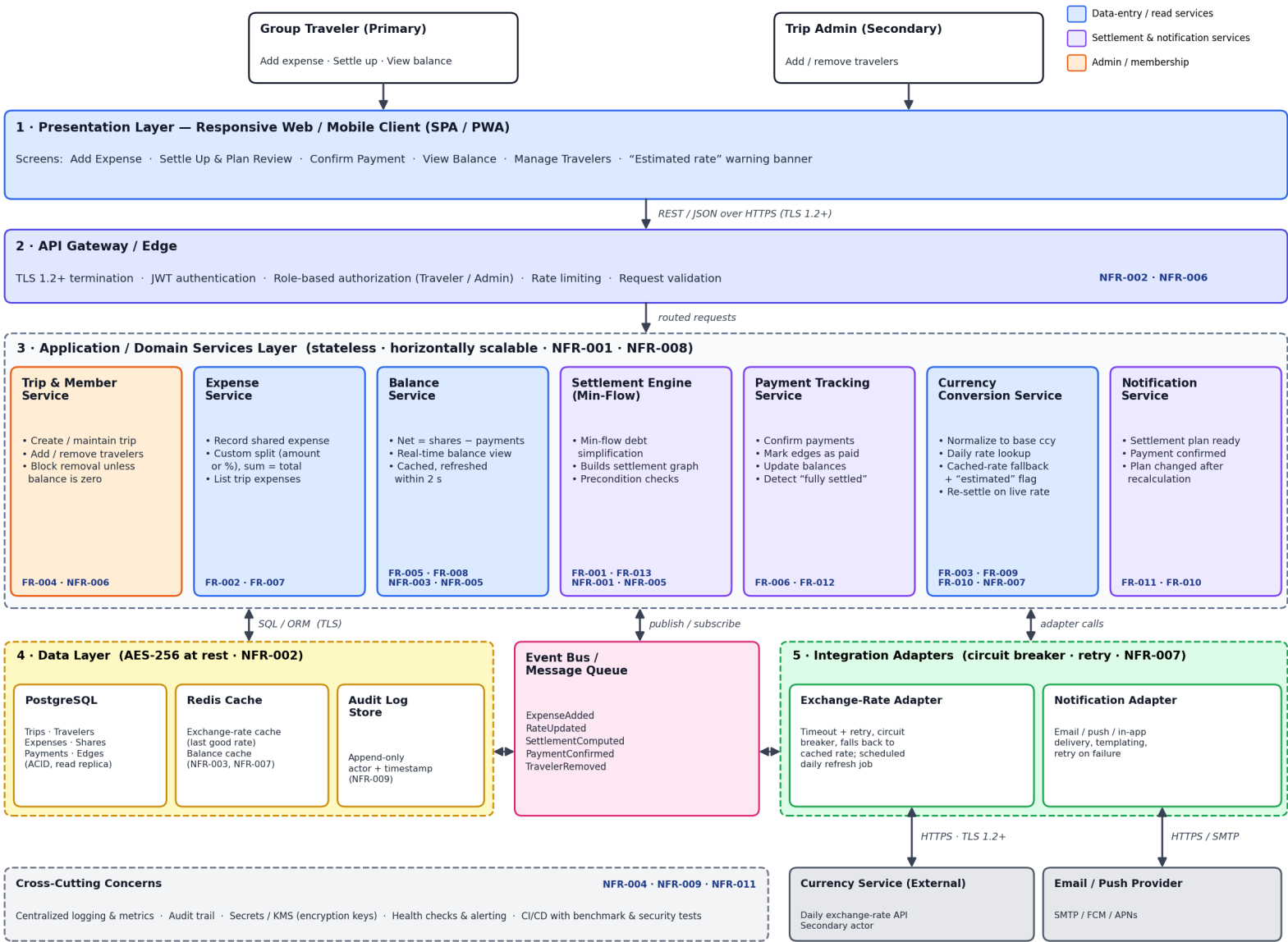
4.5 Business rules

ID	Rule
BR-01	The sum of all shares of an expense equals the expense's base-currency total; any rounding residual is allocated by the documented rounding rule (proposed: round half to even at the minor unit, residual cent assigned to the largest share).
BR-02	The sum of all net balances in a trip is zero.
BR-03	Net balance = amounts paid – owed shares, adjusted by confirmed settlement payments.
BR-04	A traveler can be removed only when their net balance is zero.
BR-05	The exchange rate used is the rate published for the expense date; cached rates mark the expense as estimated until recalculated with a live rate.
BR-06	Settlement requires at least two travelers and at least one expense.
BR-07	A trip is "Settled" only when all settlement edges are Paid and all balances are zero.
BR-08	Only a Trip Admin may add or remove travelers; users can access only trips they belong to.

5. System Architecture

Trip Expense Simplification Engine — System Architecture (Problem Statement #37)

Layered, service-oriented architecture · FR / NFR tags on each component trace to the Requirements Traceability Matrix



5.1 Architecture summary

- **Layers:** responsive client → API gateway (TLS, JWT, RBAC, rate limiting) → seven stateless domain services → data layer (PostgreSQL, Redis, audit store), event bus and integration adapters → external Currency Service and messaging provider.
- **Settlement Engine** is a pure module (net balances in, payment edges out), which keeps it fast (NFR-001) and independently testable (NFR-011).
- **Resilience:** the Exchange-Rate Adapter applies timeout, retry and a circuit breaker; Redis holds the last good rate so settlement continues in “estimated” mode (FR-009, NFR-007).
- **Events:** RateUpdated triggers re-settlement (FR-010); SettlementComputed and PaymentConfirmed drive notifications (FR-011) without blocking user requests.
- **Data:** PostgreSQL is the system of record (ACID); Redis caches rates and balances only, so losing it affects speed, not correctness.
- **Security:** TLS 1.2+ everywhere, AES-256 at rest with KMS-managed keys, role-based authorization, append-only audit log (NFR-002, 006, 009).

6. Requirements Traceability Matrix

Req ID	Use case	Architecture component(s)	Test case	Verification
FR-001	Compute Debt Settlement	Settlement Engine; PostgreSQL	TC-FR-001	Test
FR-002	Add Expense	Web Client; API Gateway; Expense Service; PostgreSQL	TC-FR-002	Test
FR-003	Convert Currency (included by Add Expense)	Currency Conversion Service; Exchange-Rate Adapter; Redis Cache	TC-FR-003	Test
FR-004	Manage Trip Travelers	Web Client; Trip & Member Service; PostgreSQL	TC-FR-004	Test
FR-005	View Balance	Web Client; Balance Service; Redis Cache	TC-FR-005	Test
FR-006	Compute Debt Settlement	Payment Tracking Service; Balance Service; PostgreSQL	TC-FR-006	Test
FR-007	Compute Debt Settlement	Web Client; Expense Service; Settlement Engine	TC-FR-007	Demonstration
FR-008	Compute Debt Settlement	Balance Service; PostgreSQL	TC-FR-008	Test
FR-009	Compute Debt Settlement	Currency Conversion Service; Exchange-Rate Adapter; Redis Cache; Web Client (banner)	TC-FR-009	Test (fault injection)
FR-010	Compute Debt Settlement; Notify Travelers	Currency Conversion Service; Settlement Engine; Event Bus; Notification Service	TC-FR-010	Test
FR-011	Notify Travelers (included by Compute Debt Settlement)	Notification Service; Event Bus; Notification Adapter	TC-FR-011	Test
FR-012	Compute Debt Settlement	Payment Tracking Service; Balance Service	TC-FR-012	Test
FR-013	Compute Debt Settlement	Settlement Engine; Expense Service; Trip & Member Service	TC-FR-013	Test
NFR-001	Compute Debt Settlement	Settlement Engine	TC-NFR-001	Performance benchmark
NFR-002	All use cases	API Gateway; PostgreSQL; Redis Cache; Adapters; KMS	TC-NFR-002	Security scan / audit
NFR-003	View Balance; Compute Debt Settlement	Balance Service; Redis Cache; API Gateway	TC-NFR-003	Load test
NFR-004	All use cases	PostgreSQL (replica, backups); stateless services; health checks	TC-NFR-004	Monitoring / DR drill
NFR-005	Add Expense; Compute Debt Settlement	Expense Service; Balance Service; Settlement Engine	TC-NFR-005	Property-based test
NFR-006	Manage Trip Travelers; all trip data access	API Gateway (authN/authZ); Trip & Member Service	TC-NFR-006	Security test
NFR-007	Convert Currency	Exchange-Rate Adapter; Redis Cache	TC-NFR-007	Fault-injection test

Req ID	Use case	Architecture component(s)	Test case	Verification
NFR-008	All use cases	Stateless services; API Gateway; Event Bus; PostgreSQL	TC-NFR-008	Load test
NFR-009	Add Expense; Manage Trip Travelers; Compute Debt Settlement	Audit Log Store; Event Bus	TC-NFR-009	Inspection / test
NFR-010	All use cases	Web / Mobile Client	TC-NFR-010	Usability + accessibility test
NFR-011	Compute Debt Settlement; Convert Currency	Settlement Engine; Currency Conversion Service; CI pipeline	TC-NFR-011	Inspection / CI report

7. Open Issues and To-Be-Determined Items

#	Issue	Why it matters / proposal
1	Minimum number of transactions is NP-hard in general.	Greedy matching gives at most $n-1$ payments and meets NFR-001; exact minimization (zero-sum group partitioning) is exponential for 50 travelers. Decide: greedy, or greedy plus bounded exact optimization for small groups (Appendix A).
2	Who may confirm a payment?	Use case says “Group Traveler”. Proposal: payee (or both parties) confirms to avoid false “paid” claims.
3	Authentication method.	Email + password, OAuth or magic link — affects gateway and UI work (WBS 5.1, 6.1).
4	Exchange-rate provider and rate basis.	Choose provider, rate type (daily close / reference) and licensing; affects FR-003, FR-009.
5	Rounding rule.	Confirm proposal in BR-01 so tests can assert exact values.
6	Expense edit / delete not specified.	Would change balances after a plan exists; define behaviour or keep out of scope.
7	Flows for Add Expense, Manage Trip Travelers, View Balance not supplied.	Section 3.2 is proposed; confirm with product owner.
8	Payment execution.	Assumed outside the system; confirm no payment-gateway integration is expected.
9	Privacy and data retention.	Confirm applicable data-protection requirements and retention beyond the 12-month audit period.

Appendix A — Settlement Algorithm (FR-001)

Approach

1. Compute each traveler's net balance in minor units (creditors positive, debtors negative); the sum must be zero.
2. Drop travelers with zero balance.
3. Match any debtor and creditor whose balances are exact opposites (a one-payment settlement).
4. Repeatedly take the largest debtor and the largest creditor, create a payment of $\min(|\text{debt}|, \text{credit})$, and reduce both balances; remove anyone who reaches zero.
5. Result: at most $n - 1$ payments for n travelers with non-zero balance, computed in $O(n \log n)$.

Optional optimization: partition travelers into independent zero-sum subgroups first; each subgroup of size k needs at most $k - 1$ payments, so more subgroups means fewer payments. Finding the maximum number of such subgroups is the NP-hard part, so limit exact search to small groups or a time budget to stay within NFR-001.

Worked example

Traveler	Net balance	Payments in plan
A	+60 (is owed)	receives 50 from C and 10 from D
B	+20 (is owed)	receives 20 from D
C	-50 (owes)	pays 50 to A
D	-30 (owes)	pays 10 to A and 20 to B

Four travelers produce 3 payments ($= n - 1$), and every balance ends at zero: A $60 - 60$, B $20 - 20$, C $-50 + 50$, D $-30 + 30$.